



VOLUME 04

RHI, VULKAN AND DIRECT3D 12

Shared explicit Render Hardware Interface, GPU resources, synchronization, Vulkan backend, Direct3D 12 backend and conformance.

DOCUMENT CODE	SKEIN-IMP-002-V04
VERSION	0.1
STATUS	DRAFT CONSTRUCTION REFERENCE
PLANNED PAGES	215
CHAPTERS	15
DATE	30 July 2026

IMPLEMENTATION MANUAL / PRINT SET

Current architecture source: SKEIN-SPEC-001 v0.4 and SKEIN-IMP-002-MASTER v0.1

04.01 RHI design principles

Purpose and boundary: explicit semantics

This page defines the purpose and boundary for RHI design principles, with particular attention to explicit semantics. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Explicit semantics is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with capabilities is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for explicit semantics; distinguish required behavior from illustrative implementation detail.
- Keep capabilities behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RhiDesignPrinciplesService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RhiDesignPrinciplesConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RhiDesignPrinciplesState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.01 and explicit semantics.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.01 RHI design principles

Architecture: capabilities

This page defines the architecture for RHI design principles, with particular attention to capabilities. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Capabilities is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with backend neutrality is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for capabilities; distinguish required behavior from illustrative implementation detail.
- Keep backend neutrality behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Rhi", "RhiDesignPrinciples");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.01 and capabilities.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.01 RHI design principles

Data model: backend neutrality

This page defines the data model for RHI design principles, with particular attention to backend neutrality. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Backend neutrality is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with no D3D11-shaped API is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for backend neutrality; distinguish required behavior from illustrative implementation detail.
- Keep no D3D11-shaped API behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RhiDesignPrinciplesService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RhiDesignPrinciplesConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RhiDesignPrinciplesState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.01 and backend neutrality.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.01 RHI design principles

Public API: no D3D11-shaped API

This page defines the public api for RHI design principles, with particular attention to no D3D11-shaped API. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

No d3d11-shaped api is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with explicit semantics is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for no D3D11-shaped API; distinguish required behavior from illustrative implementation detail.
- Keep explicit semantics behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Rhi", "RhiDesignPrinciples");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.01 and no D3D11-shaped API.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.01 RHI design principles

Ownership and lifetime: explicit semantics

This page defines the ownership and lifetime for RHI design principles, with particular attention to explicit semantics. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Explicit semantics is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with capabilities is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for explicit semantics; distinguish required behavior from illustrative implementation detail.
- Keep capabilities behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RhiDesignPrinciplesService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RhiDesignPrinciplesConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RhiDesignPrinciplesState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.01 and explicit semantics.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.01 RHI design principles

Threading model: capabilities

This page defines the threading model for RHI design principles, with particular attention to capabilities. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Capabilities is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with backend neutrality is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for capabilities; distinguish required behavior from illustrative implementation detail.
- Keep backend neutrality behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Rhi", "RhiDesignPrinciples");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.01 and capabilities.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.01 RHI design principles

Memory strategy: backend neutrality

This page defines the memory strategy for RHI design principles, with particular attention to backend neutrality. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Backend neutrality is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with no D3D11-shaped API is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for backend neutrality; distinguish required behavior from illustrative implementation detail.
- Keep no D3D11-shaped API behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RhiDesignPrinciplesService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RhiDesignPrinciplesConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RhiDesignPrinciplesState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.01 and backend neutrality.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.01 RHI design principles

Failure handling: no D3D11-shaped API

This page defines the failure handling for RHI design principles, with particular attention to no D3D11-shaped API. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

No d3d11-shaped api is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with explicit semantics is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for no D3D11-shaped API; distinguish required behavior from illustrative implementation detail.
- Keep explicit semantics behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Rhi", "RhiDesignPrinciples");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.01 and no D3D11-shaped API.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.01 RHI design principles

Diagnostics: explicit semantics

This page defines the diagnostics for RHI design principles, with particular attention to explicit semantics. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Explicit semantics is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with capabilities is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for explicit semantics; distinguish required behavior from illustrative implementation detail.
- Keep capabilities behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RhiDesignPrinciplesService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RhiDesignPrinciplesConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RhiDesignPrinciplesState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.01 and explicit semantics.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.01 RHI design principles

Implementation sequence: capabilities

This page defines the implementation sequence for RHI design principles, with particular attention to capabilities. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Capabilities is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with backend neutrality is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for capabilities; distinguish required behavior from illustrative implementation detail.
- Keep backend neutrality behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Rhi", "RhiDesignPrinciples");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.01 and capabilities.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.02 Adapters, devices and queues

Purpose and boundary: selection

This page defines the purpose and boundary for Adapters, devices and queues, with particular attention to selection. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Selection is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with feature tiers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for selection; distinguish required behavior from illustrative implementation detail.
- Keep feature tiers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AdaptersDevicesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AdaptersDevicesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AdaptersDevicesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.02 and selection.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.02 Adapters, devices and queues

Architecture: feature tiers

This page defines the architecture for Adapters, devices and queues, with particular attention to feature tiers. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Feature tiers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with queue families is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for feature tiers; distinguish required behavior from illustrative implementation detail.
- Keep queue families behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Adapters", "AdaptersDevicesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.02 and feature tiers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.02 Adapters, devices and queues

Data model: queue families

This page defines the data model for Adapters, devices and queues, with particular attention to queue families. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Queue families is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with device removal is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for queue families; distinguish required behavior from illustrative implementation detail.
- Keep device removal behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AdaptersDevicesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AdaptersDevicesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AdaptersDevicesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.02 and queue families.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.02 Adapters, devices and queues

Public API: device removal

This page defines the public api for Adapters, devices and queues, with particular attention to device removal. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Device removal is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with selection is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for device removal; distinguish required behavior from illustrative implementation detail.
- Keep selection behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Adapters", "AdaptersDevicesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.02 and device removal.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.02 Adapters, devices and queues

Ownership and lifetime: selection

This page defines the ownership and lifetime for Adapters, devices and queues, with particular attention to selection. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Selection is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with feature tiers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for selection; distinguish required behavior from illustrative implementation detail.
- Keep feature tiers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AdaptersDevicesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AdaptersDevicesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AdaptersDevicesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.02 and selection.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.02 Adapters, devices and queues

Threading model: feature tiers

This page defines the threading model for Adapters, devices and queues, with particular attention to feature tiers. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Feature tiers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with queue families is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for feature tiers; distinguish required behavior from illustrative implementation detail.
- Keep queue families behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Adapters", "AdaptersDevicesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.02 and feature tiers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.02 Adapters, devices and queues

Memory strategy: queue families

This page defines the memory strategy for Adapters, devices and queues, with particular attention to queue families. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Queue families is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with device removal is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for queue families; distinguish required behavior from illustrative implementation detail.
- Keep device removal behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AdaptersDevicesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AdaptersDevicesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AdaptersDevicesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.02 and queue families.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.02 Adapters, devices and queues

Failure handling: device removal

This page defines the failure handling for Adapters, devices and queues, with particular attention to device removal. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Device removal is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with selection is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for device removal; distinguish required behavior from illustrative implementation detail.
- Keep selection behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Adapters", "AdaptersDevicesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.02 and device removal.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.02 Adapters, devices and queues

Diagnostics: selection

This page defines the diagnostics for Adapters, devices and queues, with particular attention to selection. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Selection is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with feature tiers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for selection; distinguish required behavior from illustrative implementation detail.
- Keep feature tiers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AdaptersDevicesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AdaptersDevicesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AdaptersDevicesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.02 and selection.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.02 Adapters, devices and queues

Implementation sequence: feature tiers

This page defines the implementation sequence for Adapters, devices and queues, with particular attention to feature tiers. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Feature tiers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with queue families is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for feature tiers; distinguish required behavior from illustrative implementation detail.
- Keep queue families behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Adapters", "AdaptersDevicesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.02 and feature tiers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.02 Adapters, devices and queues

Verification: queue families

This page defines the verification for Adapters, devices and queues, with particular attention to queue families. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Queue families is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with device removal is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for queue families; distinguish required behavior from illustrative implementation detail.
- Keep device removal behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AdaptersDevicesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AdaptersDevicesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AdaptersDevicesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.02 and queue families.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.02 Adapters, devices and queues

Completion gate: device removal

This page defines the completion gate for Adapters, devices and queues, with particular attention to device removal. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Device removal is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with selection is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for device removal; distinguish required behavior from illustrative implementation detail.
- Keep selection behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Adapters", "AdaptersDevicesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.02 and device removal.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.03 Resource descriptors and handles

Purpose and boundary: buffers

This page defines the purpose and boundary for Resource descriptors and handles, with particular attention to buffers. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Buffers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with textures is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for buffers; distinguish required behavior from illustrative implementation detail.
- Keep textures behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ResourceDescriptorsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ResourceDescriptorsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ResourceDescriptorsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.03 and buffers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.03 Resource descriptors and handles

Architecture: textures

This page defines the architecture for Resource descriptors and handles, with particular attention to textures. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Textures is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with views is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for textures; distinguish required behavior from illustrative implementation detail.
- Keep views behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Resource", "ResourceDescriptorsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.03 and textures.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.03 Resource descriptors and handles

Data model: views

This page defines the data model for Resource descriptors and handles, with particular attention to views. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Views is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with samplers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for views; distinguish required behavior from illustrative implementation detail.
- Keep samplers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ResourceDescriptorsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ResourceDescriptorsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ResourceDescriptorsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.03 and views.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.03 Resource descriptors and handles

Public API: samplers

This page defines the public api for Resource descriptors and handles, with particular attention to samplers. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Samplers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with lifetimes is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for samplers; distinguish required behavior from illustrative implementation detail.
- Keep lifetimes behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Resource", "ResourceDescriptorsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.03 and samplers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.03 Resource descriptors and handles

Ownership and lifetime: lifetimes

This page defines the ownership and lifetime for Resource descriptors and handles, with particular attention to lifetimes. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Lifetimes is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with debug names is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for lifetimes; distinguish required behavior from illustrative implementation detail.
- Keep debug names behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ResourceDescriptorsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ResourceDescriptorsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ResourceDescriptorsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.03 and lifetimes.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.03 Resource descriptors and handles

Threading model: debug names

This page defines the threading model for Resource descriptors and handles, with particular attention to debug names. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Debug names is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with buffers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for debug names; distinguish required behavior from illustrative implementation detail.
- Keep buffers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Resource", "ResourceDescriptorsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.03 and debug names.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.03 Resource descriptors and handles

Memory strategy: buffers

This page defines the memory strategy for Resource descriptors and handles, with particular attention to buffers. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Buffers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with textures is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for buffers; distinguish required behavior from illustrative implementation detail.
- Keep textures behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ResourceDescriptorsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ResourceDescriptorsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ResourceDescriptorsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.03 and buffers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.03 Resource descriptors and handles

Failure handling: textures

This page defines the failure handling for Resource descriptors and handles, with particular attention to textures. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Textures is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with views is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for textures; distinguish required behavior from illustrative implementation detail.
- Keep views behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Resource", "ResourceDescriptorsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.03 and textures.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.03 Resource descriptors and handles

Diagnostics: views

This page defines the diagnostics for Resource descriptors and handles, with particular attention to views. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Views is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with samplers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for views; distinguish required behavior from illustrative implementation detail.
- Keep samplers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ResourceDescriptorsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ResourceDescriptorsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ResourceDescriptorsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.03 and views.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.03 Resource descriptors and handles

Implementation sequence: samplers

This page defines the implementation sequence for Resource descriptors and handles, with particular attention to samplers. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Samplers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with lifetimes is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for samplers; distinguish required behavior from illustrative implementation detail.
- Keep lifetimes behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Resource", "ResourceDescriptorsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.03 and samplers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.03 Resource descriptors and handles

Verification: lifetimes

This page defines the verification for Resource descriptors and handles, with particular attention to lifetimes. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Lifetimes is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with debug names is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for lifetimes; distinguish required behavior from illustrative implementation detail.
- Keep debug names behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ResourceDescriptorsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ResourceDescriptorsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ResourceDescriptorsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.03 and lifetimes.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.03 Resource descriptors and handles

Completion gate: debug names

This page defines the completion gate for Resource descriptors and handles, with particular attention to debug names. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Debug names is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with buffers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for debug names; distinguish required behavior from illustrative implementation detail.
- Keep buffers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Resource", "ResourceDescriptorsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.03 and debug names.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.03 Resource descriptors and handles

Purpose and boundary: buffers

This page defines the purpose and boundary for Resource descriptors and handles, with particular attention to buffers. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Buffers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with textures is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for buffers; distinguish required behavior from illustrative implementation detail.
- Keep textures behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ResourceDescriptorsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ResourceDescriptorsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ResourceDescriptorsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.03 and buffers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.03 Resource descriptors and handles

Architecture: textures

This page defines the architecture for Resource descriptors and handles, with particular attention to textures. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Textures is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with views is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for textures; distinguish required behavior from illustrative implementation detail.
- Keep views behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Resource", "ResourceDescriptorsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.03 and textures.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.03 Resource descriptors and handles

Data model: views

This page defines the data model for Resource descriptors and handles, with particular attention to views. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Views is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with samplers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for views; distinguish required behavior from illustrative implementation detail.
- Keep samplers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ResourceDescriptorsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ResourceDescriptorsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ResourceDescriptorsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.03 and views.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.03 Resource descriptors and handles

Public API: samplers

This page defines the public api for Resource descriptors and handles, with particular attention to samplers. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Samplers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with lifetimes is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for samplers; distinguish required behavior from illustrative implementation detail.
- Keep lifetimes behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Resource", "ResourceDescriptorsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.03 and samplers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.04 Command allocators and command lists

Purpose and boundary: recording model

This page defines the purpose and boundary for Command allocators and command lists, with particular attention to recording model. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Recording model is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with threading is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for recording model; distinguish required behavior from illustrative implementation detail.
- Keep threading behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CommandAllocatorsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CommandAllocatorsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CommandAllocatorsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.04 and recording model.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.04 Command allocators and command lists

Architecture: threading

This page defines the architecture for Command allocators and command lists, with particular attention to threading. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Threading is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with reset rules is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for threading; distinguish required behavior from illustrative implementation detail.
- Keep reset rules behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Command", "CommandAllocatorsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.04 and threading.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.04 Command allocators and command lists

Data model: reset rules

This page defines the data model for Command allocators and command lists, with particular attention to reset rules. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Reset rules is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with submission is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for reset rules; distinguish required behavior from illustrative implementation detail.
- Keep submission behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CommandAllocatorsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CommandAllocatorsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CommandAllocatorsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.04 and reset rules.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.04 Command allocators and command lists

Public API: submission

This page defines the public api for Command allocators and command lists, with particular attention to submission. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Submission is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with recording model is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for submission; distinguish required behavior from illustrative implementation detail.
- Keep recording model behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Command", "CommandAllocatorsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.04 and submission.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.04 Command allocators and command lists

Ownership and lifetime: recording model

This page defines the ownership and lifetime for Command allocators and command lists, with particular attention to recording model. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Recording model is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with threading is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for recording model; distinguish required behavior from illustrative implementation detail.
- Keep threading behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CommandAllocatorsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CommandAllocatorsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CommandAllocatorsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.04 and recording model.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.04 Command allocators and command lists

Threading model: threading

This page defines the threading model for Command allocators and command lists, with particular attention to threading. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Threading is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with reset rules is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for threading; distinguish required behavior from illustrative implementation detail.
- Keep reset rules behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Command", "CommandAllocatorsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.04 and threading.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.04 Command allocators and command lists

Memory strategy: reset rules

This page defines the memory strategy for Command allocators and command lists, with particular attention to reset rules. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Reset rules is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with submission is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for reset rules; distinguish required behavior from illustrative implementation detail.
- Keep submission behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CommandAllocatorsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CommandAllocatorsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CommandAllocatorsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.04 and reset rules.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.04 Command allocators and command lists

Failure handling: submission

This page defines the failure handling for Command allocators and command lists, with particular attention to submission. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Submission is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with recording model is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for submission; distinguish required behavior from illustrative implementation detail.
- Keep recording model behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Command", "CommandAllocatorsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.04 and submission.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.04 Command allocators and command lists

Diagnostics: recording model

This page defines the diagnostics for Command allocators and command lists, with particular attention to recording model. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Recording model is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with threading is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for recording model; distinguish required behavior from illustrative implementation detail.
- Keep threading behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CommandAllocatorsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CommandAllocatorsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CommandAllocatorsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.04 and recording model.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.04 Command allocators and command lists

Implementation sequence: threading

This page defines the implementation sequence for Command allocators and command lists, with particular attention to threading. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Threading is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with reset rules is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for threading; distinguish required behavior from illustrative implementation detail.
- Keep reset rules behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Command", "CommandAllocatorsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.04 and threading.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.04 Command allocators and command lists

Verification: reset rules

This page defines the verification for Command allocators and command lists, with particular attention to reset rules. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Reset rules is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with submission is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for reset rules; distinguish required behavior from illustrative implementation detail.
- Keep submission behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CommandAllocatorsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CommandAllocatorsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CommandAllocatorsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.04 and reset rules.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.04 Command allocators and command lists

Completion gate: submission

This page defines the completion gate for Command allocators and command lists, with particular attention to submission. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Submission is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with recording model is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for submission; distinguish required behavior from illustrative implementation detail.
- Keep recording model behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Command", "CommandAllocatorsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.04 and submission.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.04 Command allocators and command lists

Purpose and boundary: recording model

This page defines the purpose and boundary for Command allocators and command lists, with particular attention to recording model. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Recording model is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with threading is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for recording model; distinguish required behavior from illustrative implementation detail.
- Keep threading behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CommandAllocatorsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CommandAllocatorsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CommandAllocatorsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.04 and recording model.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.04 Command allocators and command lists

Architecture: threading

This page defines the architecture for Command allocators and command lists, with particular attention to threading. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Threading is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with reset rules is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for threading; distinguish required behavior from illustrative implementation detail.
- Keep reset rules behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Command", "CommandAllocatorsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.04 and threading.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.05 Resource states and barriers

Purpose and boundary: state model

This page defines the purpose and boundary for Resource states and barriers, with particular attention to state model. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

State model is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with transitions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for state model; distinguish required behavior from illustrative implementation detail.
- Keep transitions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ResourceStatesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ResourceStatesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ResourceStatesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.05 and state model.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.05 Resource states and barriers

Architecture: transitions

This page defines the architecture for Resource states and barriers, with particular attention to transitions. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Transitions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with ownership transfer is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for transitions; distinguish required behavior from illustrative implementation detail.
- Keep ownership transfer behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Resource", "ResourceStatesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.05 and transitions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.05 Resource states and barriers

Data model: ownership transfer

This page defines the data model for Resource states and barriers, with particular attention to ownership transfer. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Ownership transfer is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with validation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for ownership transfer; distinguish required behavior from illustrative implementation detail.
- Keep validation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ResourceStatesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ResourceStatesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ResourceStatesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.05 and ownership transfer.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.05 Resource states and barriers

Public API: validation

This page defines the public api for Resource states and barriers, with particular attention to validation. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Validation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with state model is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for validation; distinguish required behavior from illustrative implementation detail.
- Keep state model behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Resource", "ResourceStatesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.05 and validation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.05 Resource states and barriers

Ownership and lifetime: state model

This page defines the ownership and lifetime for Resource states and barriers, with particular attention to state model. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

State model is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with transitions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for state model; distinguish required behavior from illustrative implementation detail.
- Keep transitions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ResourceStatesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ResourceStatesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ResourceStatesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.05 and state model.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.05 Resource states and barriers

Threading model: transitions

This page defines the threading model for Resource states and barriers, with particular attention to transitions. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Transitions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with ownership transfer is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for transitions; distinguish required behavior from illustrative implementation detail.
- Keep ownership transfer behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Resource", "ResourceStatesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.05 and transitions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.05 Resource states and barriers

Memory strategy: ownership transfer

This page defines the memory strategy for Resource states and barriers, with particular attention to ownership transfer. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Ownership transfer is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with validation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for ownership transfer; distinguish required behavior from illustrative implementation detail.
- Keep validation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ResourceStatesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ResourceStatesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ResourceStatesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.05 and ownership transfer.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.05 Resource states and barriers

Failure handling: validation

This page defines the failure handling for Resource states and barriers, with particular attention to validation. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Validation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with state model is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for validation; distinguish required behavior from illustrative implementation detail.
- Keep state model behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Resource", "ResourceStatesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.05 and validation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.05 Resource states and barriers

Diagnostics: state model

This page defines the diagnostics for Resource states and barriers, with particular attention to state model. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

State model is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with transitions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for state model; distinguish required behavior from illustrative implementation detail.
- Keep transitions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ResourceStatesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ResourceStatesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ResourceStatesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.05 and state model.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.05 Resource states and barriers

Implementation sequence: transitions

This page defines the implementation sequence for Resource states and barriers, with particular attention to transitions. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Transitions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with ownership transfer is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for transitions; distinguish required behavior from illustrative implementation detail.
- Keep ownership transfer behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Resource", "ResourceStatesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.05 and transitions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.05 Resource states and barriers

Verification: ownership transfer

This page defines the verification for Resource states and barriers, with particular attention to ownership transfer. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Ownership transfer is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with validation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for ownership transfer; distinguish required behavior from illustrative implementation detail.
- Keep validation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ResourceStatesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ResourceStatesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ResourceStatesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.05 and ownership transfer.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.05 Resource states and barriers

Completion gate: validation

This page defines the completion gate for Resource states and barriers, with particular attention to validation. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Validation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with state model is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for validation; distinguish required behavior from illustrative implementation detail.
- Keep state model behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Resource", "ResourceStatesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.05 and validation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.05 Resource states and barriers

Purpose and boundary: state model

This page defines the purpose and boundary for Resource states and barriers, with particular attention to state model. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

State model is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with transitions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for state model; distinguish required behavior from illustrative implementation detail.
- Keep transitions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ResourceStatesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ResourceStatesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ResourceStatesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.05 and state model.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.05 Resource states and barriers

Architecture: transitions

This page defines the architecture for Resource states and barriers, with particular attention to transitions. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Transitions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with ownership transfer is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for transitions; distinguish required behavior from illustrative implementation detail.
- Keep ownership transfer behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Resource", "ResourceStatesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.05 and transitions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.05 Resource states and barriers

Data model: ownership transfer

This page defines the data model for Resource states and barriers, with particular attention to ownership transfer. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Ownership transfer is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with validation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for ownership transfer; distinguish required behavior from illustrative implementation detail.
- Keep validation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ResourceStatesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ResourceStatesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ResourceStatesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.05 and ownership transfer.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.06 Descriptors and binding model

Purpose and boundary: bindless direction

This page defines the purpose and boundary for Descriptors and binding model, with particular attention to bindless direction. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Bindless direction is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with descriptor heaps/sets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for bindless direction; distinguish required behavior from illustrative implementation detail.
- Keep descriptor heaps/sets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DescriptorsAndBindingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DescriptorsAndBindingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DescriptorsAndBindingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.06 and bindless direction.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.06 Descriptors and binding model

Architecture: descriptor heaps/sets

This page defines the architecture for Descriptors and binding model, with particular attention to descriptor heaps/sets. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Descriptor heaps/sets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with updates is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for descriptor heaps/sets; distinguish required behavior from illustrative implementation detail.
- Keep updates behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Descriptors", "DescriptorsAndBinding");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.06 and descriptor heaps/sets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.06 Descriptors and binding model

Data model: updates

This page defines the data model for Descriptors and binding model, with particular attention to updates. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Updates is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with limits is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for updates; distinguish required behavior from illustrative implementation detail.
- Keep limits behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DescriptorsAndBindingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DescriptorsAndBindingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DescriptorsAndBindingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.06 and updates.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.06 Descriptors and binding model

Public API: limits

This page defines the public api for Descriptors and binding model, with particular attention to limits. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Limits is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with bindless direction is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for limits; distinguish required behavior from illustrative implementation detail.
- Keep bindless direction behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Descriptors", "DescriptorsAndBinding");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.06 and limits.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.06 Descriptors and binding model

Ownership and lifetime: bindless direction

This page defines the ownership and lifetime for Descriptors and binding model, with particular attention to bindless direction. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Bindless direction is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with descriptor heaps/sets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for bindless direction; distinguish required behavior from illustrative implementation detail.
- Keep descriptor heaps/sets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DescriptorsAndBindingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DescriptorsAndBindingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DescriptorsAndBindingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.06 and bindless direction.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.06 Descriptors and binding model

Threading model: descriptor heaps/sets

This page defines the threading model for Descriptors and binding model, with particular attention to descriptor heaps/sets. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Descriptor heaps/sets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with updates is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for descriptor heaps/sets; distinguish required behavior from illustrative implementation detail.
- Keep updates behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Descriptors", "DescriptorsAndBinding");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.06 and descriptor heaps/sets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.06 Descriptors and binding model

Memory strategy: updates

This page defines the memory strategy for Descriptors and binding model, with particular attention to updates. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Updates is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with limits is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for updates; distinguish required behavior from illustrative implementation detail.
- Keep limits behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DescriptorsAndBindingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DescriptorsAndBindingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DescriptorsAndBindingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.06 and updates.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.06 Descriptors and binding model

Failure handling: limits

This page defines the failure handling for Descriptors and binding model, with particular attention to limits. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Limits is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with bindless direction is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for limits; distinguish required behavior from illustrative implementation detail.
- Keep bindless direction behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Descriptors", "DescriptorsAndBinding");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.06 and limits.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.06 Descriptors and binding model

Diagnostics: bindless direction

This page defines the diagnostics for Descriptors and binding model, with particular attention to bindless direction. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Bindless direction is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with descriptor heaps/sets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for bindless direction; distinguish required behavior from illustrative implementation detail.
- Keep descriptor heaps/sets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DescriptorsAndBindingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DescriptorsAndBindingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DescriptorsAndBindingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.06 and bindless direction.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.06 Descriptors and binding model

Implementation sequence: descriptor heaps/sets

This page defines the implementation sequence for Descriptors and binding model, with particular attention to descriptor heaps/sets. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Descriptor heaps/sets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with updates is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for descriptor heaps/sets; distinguish required behavior from illustrative implementation detail.
- Keep updates behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Descriptors", "DescriptorsAndBinding");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.06 and descriptor heaps/sets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.06 Descriptors and binding model

Verification: updates

This page defines the verification for Descriptors and binding model, with particular attention to updates. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Updates is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with limits is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for updates; distinguish required behavior from illustrative implementation detail.
- Keep limits behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DescriptorsAndBindingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DescriptorsAndBindingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DescriptorsAndBindingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.06 and updates.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.06 Descriptors and binding model

Completion gate: limits

This page defines the completion gate for Descriptors and binding model, with particular attention to limits. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Limits is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with bindless direction is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for limits; distinguish required behavior from illustrative implementation detail.
- Keep bindless direction behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Descriptors", "DescriptorsAndBinding");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.06 and limits.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.06 Descriptors and binding model

Purpose and boundary: bindless direction

This page defines the purpose and boundary for Descriptors and binding model, with particular attention to bindless direction. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Bindless direction is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with descriptor heaps/sets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for bindless direction; distinguish required behavior from illustrative implementation detail.
- Keep descriptor heaps/sets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DescriptorsAndBindingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DescriptorsAndBindingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DescriptorsAndBindingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.06 and bindless direction.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.06 Descriptors and binding model

Architecture: descriptor heaps/sets

This page defines the architecture for Descriptors and binding model, with particular attention to descriptor heaps/sets. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Descriptor heaps/sets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with updates is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for descriptor heaps/sets; distinguish required behavior from illustrative implementation detail.
- Keep updates behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Descriptors", "DescriptorsAndBinding");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.06 and descriptor heaps/sets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.06 Descriptors and binding model

Data model: updates

This page defines the data model for Descriptors and binding model, with particular attention to updates. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Updates is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with limits is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for updates; distinguish required behavior from illustrative implementation detail.
- Keep limits behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DescriptorsAndBindingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DescriptorsAndBindingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DescriptorsAndBindingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.06 and updates.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.07 Pipeline and shader objects

Purpose and boundary: graphics

This page defines the purpose and boundary for Pipeline and shader objects, with particular attention to graphics. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Graphics is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with compute is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for graphics; distinguish required behavior from illustrative implementation detail.
- Keep compute behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PipelineAndShaderService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PipelineAndShaderConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PipelineAndShaderState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.07 and graphics.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.07 Pipeline and shader objects

Architecture: compute

This page defines the architecture for Pipeline and shader objects, with particular attention to compute. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Compute is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with pipeline cache is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for compute; distinguish required behavior from illustrative implementation detail.
- Keep pipeline cache behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Pipeline", "PipelineAndShader");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.07 and compute.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.07 Pipeline and shader objects

Data model: pipeline cache

This page defines the data model for Pipeline and shader objects, with particular attention to pipeline cache. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Pipeline cache is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with specialization is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for pipeline cache; distinguish required behavior from illustrative implementation detail.
- Keep specialization behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PipelineAndShaderService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PipelineAndShaderConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PipelineAndShaderState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.07 and pipeline cache.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.07 Pipeline and shader objects

Public API: specialization

This page defines the public api for Pipeline and shader objects, with particular attention to specialization. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Specialization is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with graphics is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for specialization; distinguish required behavior from illustrative implementation detail.
- Keep graphics behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Pipeline", "PipelineAndShader");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.07 and specialization.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.07 Pipeline and shader objects

Ownership and lifetime: graphics

This page defines the ownership and lifetime for Pipeline and shader objects, with particular attention to graphics. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Graphics is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with compute is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for graphics; distinguish required behavior from illustrative implementation detail.
- Keep compute behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PipelineAndShaderService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PipelineAndShaderConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PipelineAndShaderState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.07 and graphics.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.07 Pipeline and shader objects

Threading model: compute

This page defines the threading model for Pipeline and shader objects, with particular attention to compute. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Compute is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with pipeline cache is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for compute; distinguish required behavior from illustrative implementation detail.
- Keep pipeline cache behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Pipeline", "PipelineAndShader");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.07 and compute.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.07 Pipeline and shader objects

Memory strategy: pipeline cache

This page defines the memory strategy for Pipeline and shader objects, with particular attention to pipeline cache. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Pipeline cache is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with specialization is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for pipeline cache; distinguish required behavior from illustrative implementation detail.
- Keep specialization behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PipelineAndShaderService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PipelineAndShaderConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PipelineAndShaderState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.07 and pipeline cache.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.07 Pipeline and shader objects

Failure handling: specialization

This page defines the failure handling for Pipeline and shader objects, with particular attention to specialization. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Specialization is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with graphics is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for specialization; distinguish required behavior from illustrative implementation detail.
- Keep graphics behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Pipeline", "PipelineAndShader");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.07 and specialization.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.07 Pipeline and shader objects

Diagnostics: graphics

This page defines the diagnostics for Pipeline and shader objects, with particular attention to graphics. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Graphics is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with compute is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for graphics; distinguish required behavior from illustrative implementation detail.
- Keep compute behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PipelineAndShaderService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PipelineAndShaderConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PipelineAndShaderState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.07 and graphics.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.07 Pipeline and shader objects

Implementation sequence: compute

This page defines the implementation sequence for Pipeline and shader objects, with particular attention to compute. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Compute is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with pipeline cache is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for compute; distinguish required behavior from illustrative implementation detail.
- Keep pipeline cache behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Pipeline", "PipelineAndShader");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.07 and compute.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.07 Pipeline and shader objects

Verification: pipeline cache

This page defines the verification for Pipeline and shader objects, with particular attention to pipeline cache. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Pipeline cache is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with specialization is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for pipeline cache; distinguish required behavior from illustrative implementation detail.
- Keep specialization behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PipelineAndShaderService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PipelineAndShaderConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PipelineAndShaderState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.07 and pipeline cache.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.07 Pipeline and shader objects

Completion gate: specialization

This page defines the completion gate for Pipeline and shader objects, with particular attention to specialization. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Specialization is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with graphics is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for specialization; distinguish required behavior from illustrative implementation detail.
- Keep graphics behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Pipeline", "PipelineAndShader");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.07 and specialization.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.07 Pipeline and shader objects

Purpose and boundary: graphics

This page defines the purpose and boundary for Pipeline and shader objects, with particular attention to graphics. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Graphics is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with compute is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for graphics; distinguish required behavior from illustrative implementation detail.
- Keep compute behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PipelineAndShaderService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PipelineAndShaderConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PipelineAndShaderState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.07 and graphics.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.07 Pipeline and shader objects

Architecture: compute

This page defines the architecture for Pipeline and shader objects, with particular attention to compute. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Compute is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with pipeline cache is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for compute; distinguish required behavior from illustrative implementation detail.
- Keep pipeline cache behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Pipeline", "PipelineAndShader");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.07 and compute.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.08 Swap chains and presentation

Purpose and boundary: formats

This page defines the purpose and boundary for Swap chains and presentation, with particular attention to formats. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Formats is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with HDR is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for formats; distinguish required behavior from illustrative implementation detail.
- Keep HDR behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SwapChainsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SwapChainsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SwapChainsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.08 and formats.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.08 Swap chains and presentation

Architecture: HDR

This page defines the architecture for Swap chains and presentation, with particular attention to HDR. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Hdr is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with resize is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for HDR; distinguish required behavior from illustrative implementation detail.
- Keep resize behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Swap", "SwapChainsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.08 and HDR.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.08 Swap chains and presentation

Data model: resize

This page defines the data model for Swap chains and presentation, with particular attention to resize. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Resize is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tearing is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for resize; distinguish required behavior from illustrative implementation detail.
- Keep tearing behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SwapChainsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SwapChainsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SwapChainsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.08 and resize.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.08 Swap chains and presentation

Public API: tearing

This page defines the public api for Swap chains and presentation, with particular attention to tearing. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Tearing is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with frame pacing is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tearing; distinguish required behavior from illustrative implementation detail.
- Keep frame pacing behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Swap", "SwapChainsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.08 and tearing.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.08 Swap chains and presentation

Ownership and lifetime: frame pacing

This page defines the ownership and lifetime for Swap chains and presentation, with particular attention to frame pacing. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Frame pacing is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with formats is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for frame pacing; distinguish required behavior from illustrative implementation detail.
- Keep formats behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SwapChainsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SwapChainsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SwapChainsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.08 and frame pacing.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.08 Swap chains and presentation

Threading model: formats

This page defines the threading model for Swap chains and presentation, with particular attention to formats. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Formats is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with HDR is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for formats; distinguish required behavior from illustrative implementation detail.
- Keep HDR behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Swap", "SwapChainsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.08 and formats.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.08 Swap chains and presentation

Memory strategy: HDR

This page defines the memory strategy for Swap chains and presentation, with particular attention to HDR. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Hdr is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with resize is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for HDR; distinguish required behavior from illustrative implementation detail.
- Keep resize behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SwapChainsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SwapChainsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SwapChainsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.08 and HDR.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.08 Swap chains and presentation

Failure handling: resize

This page defines the failure handling for Swap chains and presentation, with particular attention to resize. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Resize is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tearing is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for resize; distinguish required behavior from illustrative implementation detail.
- Keep tearing behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Swap", "SwapChainsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.08 and resize.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.08 Swap chains and presentation

Diagnostics: tearing

This page defines the diagnostics for Swap chains and presentation, with particular attention to tearing. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Tearing is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with frame pacing is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tearing; distinguish required behavior from illustrative implementation detail.
- Keep frame pacing behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SwapChainsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SwapChainsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SwapChainsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.08 and tearing.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.08 Swap chains and presentation

Implementation sequence: frame pacing

This page defines the implementation sequence for Swap chains and presentation, with particular attention to frame pacing. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Frame pacing is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with formats is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for frame pacing; distinguish required behavior from illustrative implementation detail.
- Keep formats behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Swap", "SwapChainsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.08 and frame pacing.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.09 GPU memory architecture

Purpose and boundary: heaps

This page defines the purpose and boundary for GPU memory architecture, with particular attention to heaps. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Heaps is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with suballocation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for heaps; distinguish required behavior from illustrative implementation detail.
- Keep suballocation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class GpuMemoryArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const GpuMemoryArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    GpuMemoryArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.09 and heaps.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.09 GPU memory architecture

Architecture: suballocation

This page defines the architecture for GPU memory architecture, with particular attention to suballocation. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Suballocation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with upload/readback is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for suballocation; distinguish required behavior from illustrative implementation detail.
- Keep upload/readback behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Gpu", "GpuMemoryArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.09 and suballocation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.09 GPU memory architecture

Data model: upload/readback

This page defines the data model for GPU memory architecture, with particular attention to upload/readback. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Upload/readback is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with budget queries is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for upload/readback; distinguish required behavior from illustrative implementation detail.
- Keep budget queries behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class GpuMemoryArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const GpuMemoryArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    GpuMemoryArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.09 and upload/readback.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.09 GPU memory architecture

Public API: budget queries

This page defines the public api for GPU memory architecture, with particular attention to budget queries. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Budget queries is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with defrag policy is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for budget queries; distinguish required behavior from illustrative implementation detail.
- Keep defrag policy behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Gpu", "GpuMemoryArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.09 and budget queries.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.09 GPU memory architecture

Ownership and lifetime: defrag policy

This page defines the ownership and lifetime for GPU memory architecture, with particular attention to defrag policy. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Defrag policy is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with heaps is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for defrag policy; distinguish required behavior from illustrative implementation detail.
- Keep heaps behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class GpuMemoryArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const GpuMemoryArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    GpuMemoryArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.09 and defrag policy.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.09 GPU memory architecture

Threading model: heaps

This page defines the threading model for GPU memory architecture, with particular attention to heaps. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Heaps is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with suballocation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for heaps; distinguish required behavior from illustrative implementation detail.
- Keep suballocation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Gpu", "GpuMemoryArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.09 and heaps.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.09 GPU memory architecture

Memory strategy: suballocation

This page defines the memory strategy for GPU memory architecture, with particular attention to suballocation. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Suballocation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with upload/readback is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for suballocation; distinguish required behavior from illustrative implementation detail.
- Keep upload/readback behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class GpuMemoryArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const GpuMemoryArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    GpuMemoryArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.09 and suballocation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.09 GPU memory architecture

Failure handling: upload/readback

This page defines the failure handling for GPU memory architecture, with particular attention to upload/readback. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Upload/readback is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with budget queries is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for upload/readback; distinguish required behavior from illustrative implementation detail.
- Keep budget queries behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Gpu", "GpuMemoryArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.09 and upload/readback.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.09 GPU memory architecture

Diagnostics: budget queries

This page defines the diagnostics for GPU memory architecture, with particular attention to budget queries. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Budget queries is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with defrag policy is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for budget queries; distinguish required behavior from illustrative implementation detail.
- Keep defrag policy behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class GpuMemoryArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const GpuMemoryArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    GpuMemoryArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.09 and budget queries.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.09 GPU memory architecture

Implementation sequence: defrag policy

This page defines the implementation sequence for GPU memory architecture, with particular attention to defrag policy. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Defrag policy is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with heaps is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for defrag policy; distinguish required behavior from illustrative implementation detail.
- Keep heaps behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Gpu", "GpuMemoryArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.09 and defrag policy.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.09 GPU memory architecture

Verification: heaps

This page defines the verification for GPU memory architecture, with particular attention to heaps. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Heaps is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with suballocation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for heaps; distinguish required behavior from illustrative implementation detail.
- Keep suballocation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class GpuMemoryArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const GpuMemoryArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    GpuMemoryArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.09 and heaps.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.09 GPU memory architecture

Completion gate: suballocation

This page defines the completion gate for GPU memory architecture, with particular attention to suballocation. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Suballocation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with upload/readback is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for suballocation; distinguish required behavior from illustrative implementation detail.
- Keep upload/readback behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Gpu", "GpuMemoryArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.09 and suballocation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.09 GPU memory architecture

Purpose and boundary: upload/readback

This page defines the purpose and boundary for GPU memory architecture, with particular attention to upload/readback. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Upload/readback is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with budget queries is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for upload/readback; distinguish required behavior from illustrative implementation detail.
- Keep budget queries behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class GpuMemoryArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const GpuMemoryArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    GpuMemoryArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.09 and upload/readback.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.09 GPU memory architecture

Architecture: budget queries

This page defines the architecture for GPU memory architecture, with particular attention to budget queries. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Budget queries is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with defrag policy is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for budget queries; distinguish required behavior from illustrative implementation detail.
- Keep defrag policy behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Gpu", "GpuMemoryArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.09 and budget queries.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.09 GPU memory architecture

Data model: defrag policy

This page defines the data model for GPU memory architecture, with particular attention to defrag policy. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Defrag policy is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with heaps is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for defrag policy; distinguish required behavior from illustrative implementation detail.
- Keep heaps behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class GpuMemoryArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const GpuMemoryArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    GpuMemoryArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.09 and defrag policy.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.10 Synchronization and frame contexts

Purpose and boundary: fences

This page defines the purpose and boundary for Synchronization and frame contexts, with particular attention to fences. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Fences is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with semaphores is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for fences; distinguish required behavior from illustrative implementation detail.
- Keep semaphores behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SynchronizationAndFrameService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SynchronizationAndFrameConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SynchronizationAndFrameState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.10 and fences.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.10 Synchronization and frame contexts

Architecture: semaphores

This page defines the architecture for Synchronization and frame contexts, with particular attention to semaphores. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Semaphores is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with timeline values is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for semaphores; distinguish required behavior from illustrative implementation detail.
- Keep timeline values behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Synchronization", "SynchronizationAndFrame");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.10 and semaphores.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.10 Synchronization and frame contexts

Data model: timeline values

This page defines the data model for Synchronization and frame contexts, with particular attention to timeline values. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Timeline values is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with frames in flight is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for timeline values; distinguish required behavior from illustrative implementation detail.
- Keep frames in flight behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SynchronizationAndFrameService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SynchronizationAndFrameConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SynchronizationAndFrameState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.10 and timeline values.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.10 Synchronization and frame contexts

Public API: frames in flight

This page defines the public api for Synchronization and frame contexts, with particular attention to frames in flight. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Frames in flight is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with fences is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for frames in flight; distinguish required behavior from illustrative implementation detail.
- Keep fences behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Synchronization", "SynchronizationAndFrame");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.10 and frames in flight.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.10 Synchronization and frame contexts

Ownership and lifetime: fences

This page defines the ownership and lifetime for Synchronization and frame contexts, with particular attention to fences. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Fences is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with semaphores is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for fences; distinguish required behavior from illustrative implementation detail.
- Keep semaphores behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SynchronizationAndFrameService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SynchronizationAndFrameConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SynchronizationAndFrameState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.10 and fences.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.10 Synchronization and frame contexts

Threading model: semaphores

This page defines the threading model for Synchronization and frame contexts, with particular attention to semaphores. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Semaphores is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with timeline values is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for semaphores; distinguish required behavior from illustrative implementation detail.
- Keep timeline values behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Synchronization", "SynchronizationAndFrame");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.10 and semaphores.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.10 Synchronization and frame contexts

Memory strategy: timeline values

This page defines the memory strategy for Synchronization and frame contexts, with particular attention to timeline values. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Timeline values is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with frames in flight is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for timeline values; distinguish required behavior from illustrative implementation detail.
- Keep frames in flight behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SynchronizationAndFrameService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SynchronizationAndFrameConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SynchronizationAndFrameState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.10 and timeline values.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.10 Synchronization and frame contexts

Failure handling: frames in flight

This page defines the failure handling for Synchronization and frame contexts, with particular attention to frames in flight. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Frames in flight is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with fences is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for frames in flight; distinguish required behavior from illustrative implementation detail.
- Keep fences behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Synchronization", "SynchronizationAndFrame");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.10 and frames in flight.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.10 Synchronization and frame contexts

Diagnostics: fences

This page defines the diagnostics for Synchronization and frame contexts, with particular attention to fences. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Fences is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with semaphores is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for fences; distinguish required behavior from illustrative implementation detail.
- Keep semaphores behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SynchronizationAndFrameService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SynchronizationAndFrameConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SynchronizationAndFrameState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.10 and fences.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.10 Synchronization and frame contexts

Implementation sequence: semaphores

This page defines the implementation sequence for Synchronization and frame contexts, with particular attention to semaphores. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Semaphores is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with timeline values is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for semaphores; distinguish required behavior from illustrative implementation detail.
- Keep timeline values behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Synchronization", "SynchronizationAndFrame");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.10 and semaphores.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.10 Synchronization and frame contexts

Verification: timeline values

This page defines the verification for Synchronization and frame contexts, with particular attention to timeline values. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Timeline values is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with frames in flight is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for timeline values; distinguish required behavior from illustrative implementation detail.
- Keep frames in flight behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SynchronizationAndFrameService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SynchronizationAndFrameConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SynchronizationAndFrameState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.10 and timeline values.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.10 Synchronization and frame contexts

Completion gate: frames in flight

This page defines the completion gate for Synchronization and frame contexts, with particular attention to frames in flight. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Frames in flight is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with fences is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for frames in flight; distinguish required behavior from illustrative implementation detail.
- Keep fences behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Synchronization", "SynchronizationAndFrame");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.10 and frames in flight.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.10 Synchronization and frame contexts

Purpose and boundary: fences

This page defines the purpose and boundary for Synchronization and frame contexts, with particular attention to fences. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Fences is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with semaphores is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for fences; distinguish required behavior from illustrative implementation detail.
- Keep semaphores behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SynchronizationAndFrameService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SynchronizationAndFrameConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SynchronizationAndFrameState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.10 and fences.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.10 Synchronization and frame contexts

Architecture: semaphores

This page defines the architecture for Synchronization and frame contexts, with particular attention to semaphores. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Semaphores is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with timeline values is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for semaphores; distinguish required behavior from illustrative implementation detail.
- Keep timeline values behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Synchronization", "SynchronizationAndFrame");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.10 and semaphores.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.11 Vulkan backend bootstrap

Purpose and boundary: instance

This page defines the purpose and boundary for Vulkan backend bootstrap, with particular attention to instance. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Instance is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with validation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for instance; distinguish required behavior from illustrative implementation detail.
- Keep validation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanBackendBootstrapService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanBackendBootstrapConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanBackendBootstrapState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.11 and instance.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.11 Vulkan backend bootstrap

Architecture: validation

This page defines the architecture for Vulkan backend bootstrap, with particular attention to validation. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Validation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with device is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for validation; distinguish required behavior from illustrative implementation detail.
- Keep device behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vulkan", "VulkanBackendBootstrap");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.11 and validation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.11 Vulkan backend bootstrap

Data model: device

This page defines the data model for Vulkan backend bootstrap, with particular attention to device. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Device is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with extensions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for device; distinguish required behavior from illustrative implementation detail.
- Keep extensions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanBackendBootstrapService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanBackendBootstrapConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanBackendBootstrapState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.11 and device.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.11 Vulkan backend bootstrap

Public API: extensions

This page defines the public api for Vulkan backend bootstrap, with particular attention to extensions. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Extensions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with surface is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for extensions; distinguish required behavior from illustrative implementation detail.
- Keep surface behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vulkan", "VulkanBackendBootstrap");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.11 and extensions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.11 Vulkan backend bootstrap

Ownership and lifetime: surface

This page defines the ownership and lifetime for Vulkan backend bootstrap, with particular attention to surface. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Surface is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with instance is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for surface; distinguish required behavior from illustrative implementation detail.
- Keep instance behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanBackendBootstrapService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanBackendBootstrapConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanBackendBootstrapState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.11 and surface.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.11 Vulkan backend bootstrap

Threading model: instance

This page defines the threading model for Vulkan backend bootstrap, with particular attention to instance. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Instance is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with validation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for instance; distinguish required behavior from illustrative implementation detail.
- Keep validation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vulkan", "VulkanBackendBootstrap");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.11 and instance.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.11 Vulkan backend bootstrap

Memory strategy: validation

This page defines the memory strategy for Vulkan backend bootstrap, with particular attention to validation. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Validation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with device is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for validation; distinguish required behavior from illustrative implementation detail.
- Keep device behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanBackendBootstrapService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanBackendBootstrapConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanBackendBootstrapState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.11 and validation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.11 Vulkan backend bootstrap

Failure handling: device

This page defines the failure handling for Vulkan backend bootstrap, with particular attention to device. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Device is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with extensions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for device; distinguish required behavior from illustrative implementation detail.
- Keep extensions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vulkan", "VulkanBackendBootstrap");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.11 and device.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.11 Vulkan backend bootstrap

Diagnostics: extensions

This page defines the diagnostics for Vulkan backend bootstrap, with particular attention to extensions. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Extensions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with surface is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for extensions; distinguish required behavior from illustrative implementation detail.
- Keep surface behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanBackendBootstrapService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanBackendBootstrapConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanBackendBootstrapState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.11 and extensions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.11 Vulkan backend bootstrap

Implementation sequence: surface

This page defines the implementation sequence for Vulkan backend bootstrap, with particular attention to surface. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Surface is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with instance is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for surface; distinguish required behavior from illustrative implementation detail.
- Keep instance behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vulkan", "VulkanBackendBootstrap");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.11 and surface.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.11 Vulkan backend bootstrap

Verification: instance

This page defines the verification for Vulkan backend bootstrap, with particular attention to instance. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Instance is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with validation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for instance; distinguish required behavior from illustrative implementation detail.
- Keep validation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanBackendBootstrapService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanBackendBootstrapConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanBackendBootstrapState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.11 and instance.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.11 Vulkan backend bootstrap

Completion gate: validation

This page defines the completion gate for Vulkan backend bootstrap, with particular attention to validation. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Validation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with device is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for validation; distinguish required behavior from illustrative implementation detail.
- Keep device behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vulkan", "VulkanBackendBootstrap");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.11 and validation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.11 Vulkan backend bootstrap

Purpose and boundary: device

This page defines the purpose and boundary for Vulkan backend bootstrap, with particular attention to device. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Device is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with extensions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for device; distinguish required behavior from illustrative implementation detail.
- Keep extensions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanBackendBootstrapService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanBackendBootstrapConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanBackendBootstrapState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.11 and device.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.11 Vulkan backend bootstrap

Architecture: extensions

This page defines the architecture for Vulkan backend bootstrap, with particular attention to extensions. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Extensions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with surface is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for extensions; distinguish required behavior from illustrative implementation detail.
- Keep surface behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vulkan", "VulkanBackendBootstrap");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.11 and extensions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Purpose and boundary: VMA decision

This page defines the purpose and boundary for Vulkan resource and command implementation, with particular attention to VMA decision. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Vma decision is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with descriptors is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for VMA decision; distinguish required behavior from illustrative implementation detail.
- Keep descriptors behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and VMA decision.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Architecture: descriptors

This page defines the architecture for Vulkan resource and command implementation, with particular attention to descriptors. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Descriptors is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with pipelines is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for descriptors; distinguish required behavior from illustrative implementation detail.
- Keep pipelines behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vulkan", "VulkanResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and descriptors.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Data model: pipelines

This page defines the data model for Vulkan resource and command implementation, with particular attention to pipelines. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Pipelines is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with barriers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for pipelines; distinguish required behavior from illustrative implementation detail.
- Keep barriers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and pipelines.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Public API: barriers

This page defines the public api for Vulkan resource and command implementation, with particular attention to barriers. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Barriers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with debug utils is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for barriers; distinguish required behavior from illustrative implementation detail.
- Keep debug utils behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vulkan", "VulkanResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and barriers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Ownership and lifetime: debug utils

This page defines the ownership and lifetime for Vulkan resource and command implementation, with particular attention to debug utils. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Debug utils is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with VMA decision is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for debug utils; distinguish required behavior from illustrative implementation detail.
- Keep VMA decision behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and debug utils.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Threading model: VMA decision

This page defines the threading model for Vulkan resource and command implementation, with particular attention to VMA decision. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Vma decision is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with descriptors is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for VMA decision; distinguish required behavior from illustrative implementation detail.
- Keep descriptors behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vulkan", "VulkanResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and VMA decision.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Memory strategy: descriptors

This page defines the memory strategy for Vulkan resource and command implementation, with particular attention to descriptors. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Descriptors is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with pipelines is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for descriptors; distinguish required behavior from illustrative implementation detail.
- Keep pipelines behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and descriptors.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Failure handling: pipelines

This page defines the failure handling for Vulkan resource and command implementation, with particular attention to pipelines. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Pipelines is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with barriers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for pipelines; distinguish required behavior from illustrative implementation detail.
- Keep barriers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vulkan", "VulkanResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and pipelines.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Diagnostics: barriers

This page defines the diagnostics for Vulkan resource and command implementation, with particular attention to barriers. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Barriers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with debug utils is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for barriers; distinguish required behavior from illustrative implementation detail.
- Keep debug utils behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and barriers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Implementation sequence: debug utils

This page defines the implementation sequence for Vulkan resource and command implementation, with particular attention to debug utils. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Debug utils is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with VMA decision is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for debug utils; distinguish required behavior from illustrative implementation detail.
- Keep VMA decision behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vulkan", "VulkanResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and debug utils.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Verification: VMA decision

This page defines the verification for Vulkan resource and command implementation, with particular attention to VMA decision. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Vma decision is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with descriptors is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for VMA decision; distinguish required behavior from illustrative implementation detail.
- Keep descriptors behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and VMA decision.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Completion gate: descriptors

This page defines the completion gate for Vulkan resource and command implementation, with particular attention to descriptors. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Descriptors is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with pipelines is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for descriptors; distinguish required behavior from illustrative implementation detail.
- Keep pipelines behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vulkan", "VulkanResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and descriptors.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Purpose and boundary: pipelines

This page defines the purpose and boundary for Vulkan resource and command implementation, with particular attention to pipelines. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Pipelines is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with barriers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for pipelines; distinguish required behavior from illustrative implementation detail.
- Keep barriers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and pipelines.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Architecture: barriers

This page defines the architecture for Vulkan resource and command implementation, with particular attention to barriers. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Barriers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with debug utils is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for barriers; distinguish required behavior from illustrative implementation detail.
- Keep debug utils behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vulkan", "VulkanResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and barriers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Data model: debug utils

This page defines the data model for Vulkan resource and command implementation, with particular attention to debug utils. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Debug utils is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with VMA decision is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for debug utils; distinguish required behavior from illustrative implementation detail.
- Keep VMA decision behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and debug utils.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Public API: VMA decision

This page defines the public api for Vulkan resource and command implementation, with particular attention to VMA decision. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Vma decision is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with descriptors is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for VMA decision; distinguish required behavior from illustrative implementation detail.
- Keep descriptors behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vulkan", "VulkanResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and VMA decision.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Ownership and lifetime: descriptors

This page defines the ownership and lifetime for Vulkan resource and command implementation, with particular attention to descriptors. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Descriptors is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with pipelines is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for descriptors; distinguish required behavior from illustrative implementation detail.
- Keep pipelines behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and descriptors.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Threading model: pipelines

This page defines the threading model for Vulkan resource and command implementation, with particular attention to pipelines. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Pipelines is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with barriers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for pipelines; distinguish required behavior from illustrative implementation detail.
- Keep barriers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vulkan", "VulkanResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and pipelines.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.12 Vulkan resource and command implementation

Memory strategy: barriers

This page defines the memory strategy for Vulkan resource and command implementation, with particular attention to barriers. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Barriers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with debug utils is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for barriers; distinguish required behavior from illustrative implementation detail.
- Keep debug utils behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VulkanResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VulkanResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VulkanResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.12 and barriers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.13 Direct3D 12 backend bootstrap

Purpose and boundary: factory

This page defines the purpose and boundary for Direct3D 12 backend bootstrap, with particular attention to factory. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Factory is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with adapter is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for factory; distinguish required behavior from illustrative implementation detail.
- Keep adapter behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dBackendBootstrapService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dBackendBootstrapConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dBackendBootstrapState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.13 and factory.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.13 Direct3D 12 backend bootstrap

Architecture: adapter

This page defines the architecture for Direct3D 12 backend bootstrap, with particular attention to adapter. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Adapter is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with device is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for adapter; distinguish required behavior from illustrative implementation detail.
- Keep device behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Direct3d", "Direct3dBackendBootstrap");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.13 and adapter.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.13 Direct3D 12 backend bootstrap

Data model: device

This page defines the data model for Direct3D 12 backend bootstrap, with particular attention to device. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Device is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with debug layer is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for device; distinguish required behavior from illustrative implementation detail.
- Keep debug layer behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dBackendBootstrapService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dBackendBootstrapConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dBackendBootstrapState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.13 and device.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.13 Direct3D 12 backend bootstrap

Public API: debug layer

This page defines the public api for Direct3D 12 backend bootstrap, with particular attention to debug layer. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Debug layer is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with DRED is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for debug layer; distinguish required behavior from illustrative implementation detail.
- Keep DRED behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Direct3d", "Direct3dBackendBootstrap");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.13 and debug layer.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.13 Direct3D 12 backend bootstrap

Ownership and lifetime: DRED

This page defines the ownership and lifetime for Direct3D 12 backend bootstrap, with particular attention to DRED. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Dred is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with factory is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for DRED; distinguish required behavior from illustrative implementation detail.
- Keep factory behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dBackendBootstrapService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dBackendBootstrapConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dBackendBootstrapState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.13 and DRED.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.13 Direct3D 12 backend bootstrap

Threading model: factory

This page defines the threading model for Direct3D 12 backend bootstrap, with particular attention to factory. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Factory is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with adapter is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for factory; distinguish required behavior from illustrative implementation detail.
- Keep adapter behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Direct3d", "Direct3dBackendBootstrap");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.13 and factory.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.13 Direct3D 12 backend bootstrap

Memory strategy: adapter

This page defines the memory strategy for Direct3D 12 backend bootstrap, with particular attention to adapter. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Adapter is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with device is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for adapter; distinguish required behavior from illustrative implementation detail.
- Keep device behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dBackendBootstrapService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dBackendBootstrapConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dBackendBootstrapState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.13 and adapter.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.13 Direct3D 12 backend bootstrap

Failure handling: device

This page defines the failure handling for Direct3D 12 backend bootstrap, with particular attention to device. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Device is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with debug layer is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for device; distinguish required behavior from illustrative implementation detail.
- Keep debug layer behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Direct3d", "Direct3dBackendBootstrap");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.13 and device.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.13 Direct3D 12 backend bootstrap

Diagnostics: debug layer

This page defines the diagnostics for Direct3D 12 backend bootstrap, with particular attention to debug layer. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Debug layer is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with DRED is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for debug layer; distinguish required behavior from illustrative implementation detail.
- Keep DRED behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dBackendBootstrapService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dBackendBootstrapConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dBackendBootstrapState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.13 and debug layer.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.13 Direct3D 12 backend bootstrap

Implementation sequence: DRED

This page defines the implementation sequence for Direct3D 12 backend bootstrap, with particular attention to DRED. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Dred is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with factory is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for DRED; distinguish required behavior from illustrative implementation detail.
- Keep factory behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Direct3d", "Direct3dBackendBootstrap");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.13 and DRED.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.13 Direct3D 12 backend bootstrap

Verification: factory

This page defines the verification for Direct3D 12 backend bootstrap, with particular attention to factory. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Factory is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with adapter is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for factory; distinguish required behavior from illustrative implementation detail.
- Keep adapter behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dBackendBootstrapService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dBackendBootstrapConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dBackendBootstrapState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.13 and factory.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.13 Direct3D 12 backend bootstrap

Completion gate: adapter

This page defines the completion gate for Direct3D 12 backend bootstrap, with particular attention to adapter. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Adapter is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with device is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for adapter; distinguish required behavior from illustrative implementation detail.
- Keep device behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Direct3d", "Direct3dBackendBootstrap");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.13 and adapter.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.13 Direct3D 12 backend bootstrap

Purpose and boundary: device

This page defines the purpose and boundary for Direct3D 12 backend bootstrap, with particular attention to device. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Device is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with debug layer is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for device; distinguish required behavior from illustrative implementation detail.
- Keep debug layer behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dBackendBootstrapService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dBackendBootstrapConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dBackendBootstrapState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.13 and device.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.13 Direct3D 12 backend bootstrap

Architecture: debug layer

This page defines the architecture for Direct3D 12 backend bootstrap, with particular attention to debug layer. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Debug layer is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with DRED is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for debug layer; distinguish required behavior from illustrative implementation detail.
- Keep DRED behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Direct3d", "Direct3dBackendBootstrap");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.13 and debug layer.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Purpose and boundary: heaps

This page defines the purpose and boundary for Direct3D 12 resource and command implementation, with particular attention to heaps. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Heaps is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with root signatures is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for heaps; distinguish required behavior from illustrative implementation detail.
- Keep root signatures behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and heaps.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Architecture: root signatures

This page defines the architecture for Direct3D 12 resource and command implementation, with particular attention to root signatures. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Root signatures is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with PSOs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for root signatures; distinguish required behavior from illustrative implementation detail.
- Keep PSOs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Direct3d", "Direct3dResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and root signatures.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Data model: PSOs

This page defines the data model for Direct3D 12 resource and command implementation, with particular attention to PSOs. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Psos is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with barriers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for PSOs; distinguish required behavior from illustrative implementation detail.
- Keep barriers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and PSOs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Public API: barriers

This page defines the public api for Direct3D 12 resource and command implementation, with particular attention to barriers. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Barriers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with PIX markers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for barriers; distinguish required behavior from illustrative implementation detail.
- Keep PIX markers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Direct3d", "Direct3dResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and barriers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Ownership and lifetime: PIX markers

This page defines the ownership and lifetime for Direct3D 12 resource and command implementation, with particular attention to PIX markers. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Pix markers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with heaps is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for PIX markers; distinguish required behavior from illustrative implementation detail.
- Keep heaps behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and PIX markers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Threading model: heaps

This page defines the threading model for Direct3D 12 resource and command implementation, with particular attention to heaps. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Heaps is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with root signatures is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for heaps; distinguish required behavior from illustrative implementation detail.
- Keep root signatures behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Direct3d", "Direct3dResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and heaps.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Memory strategy: root signatures

This page defines the memory strategy for Direct3D 12 resource and command implementation, with particular attention to root signatures. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Root signatures is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with PSOs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for root signatures; distinguish required behavior from illustrative implementation detail.
- Keep PSOs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and root signatures.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Failure handling: PSOs

This page defines the failure handling for Direct3D 12 resource and command implementation, with particular attention to PSOs. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Psos is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with barriers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for PSOs; distinguish required behavior from illustrative implementation detail.
- Keep barriers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Direct3d", "Direct3dResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and PSOs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Diagnostics: barriers

This page defines the diagnostics for Direct3D 12 resource and command implementation, with particular attention to barriers. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Barriers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with PIX markers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for barriers; distinguish required behavior from illustrative implementation detail.
- Keep PIX markers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and barriers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Implementation sequence: PIX markers

This page defines the implementation sequence for Direct3D 12 resource and command implementation, with particular attention to PIX markers. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Pix markers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with heaps is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for PIX markers; distinguish required behavior from illustrative implementation detail.
- Keep heaps behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Direct3d", "Direct3dResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and PIX markers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Verification: heaps

This page defines the verification for Direct3D 12 resource and command implementation, with particular attention to heaps. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Heaps is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with root signatures is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for heaps; distinguish required behavior from illustrative implementation detail.
- Keep root signatures behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and heaps.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Completion gate: root signatures

This page defines the completion gate for Direct3D 12 resource and command implementation, with particular attention to root signatures. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Root signatures is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with PSOs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for root signatures; distinguish required behavior from illustrative implementation detail.
- Keep PSOs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Direct3d", "Direct3dResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and root signatures.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Purpose and boundary: PSOs

This page defines the purpose and boundary for Direct3D 12 resource and command implementation, with particular attention to PSOs. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Psos is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with barriers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for PSOs; distinguish required behavior from illustrative implementation detail.
- Keep barriers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and PSOs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Architecture: barriers

This page defines the architecture for Direct3D 12 resource and command implementation, with particular attention to barriers. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Barriers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with PIX markers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for barriers; distinguish required behavior from illustrative implementation detail.
- Keep PIX markers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Direct3d", "Direct3dResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and barriers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Data model: PIX markers

This page defines the data model for Direct3D 12 resource and command implementation, with particular attention to PIX markers. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Pix markers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with heaps is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for PIX markers; distinguish required behavior from illustrative implementation detail.
- Keep heaps behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and PIX markers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Public API: heaps

This page defines the public api for Direct3D 12 resource and command implementation, with particular attention to heaps. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Heaps is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with root signatures is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for heaps; distinguish required behavior from illustrative implementation detail.
- Keep root signatures behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Direct3d", "Direct3dResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and heaps.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Ownership and lifetime: root signatures

This page defines the ownership and lifetime for Direct3D 12 resource and command implementation, with particular attention to root signatures. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Root signatures is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with PSOs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for root signatures; distinguish required behavior from illustrative implementation detail.
- Keep PSOs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and root signatures.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Threading model: PSOs

This page defines the threading model for Direct3D 12 resource and command implementation, with particular attention to PSOs. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Psos is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with barriers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for PSOs; distinguish required behavior from illustrative implementation detail.
- Keep barriers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Direct3d", "Direct3dResourceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and PSOs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.14 Direct3D 12 resource and command implementation

Memory strategy: barriers

This page defines the memory strategy for Direct3D 12 resource and command implementation, with particular attention to barriers. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Barriers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with PIX markers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for barriers; distinguish required behavior from illustrative implementation detail.
- Keep PIX markers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Direct3dResourceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Direct3dResourceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Direct3dResourceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.14 and barriers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.15 RHI conformance and backend parity

Purpose and boundary: shared tests

This page defines the purpose and boundary for RHI conformance and backend parity, with particular attention to shared tests. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Shared tests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with capture tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for shared tests; distinguish required behavior from illustrative implementation detail.
- Keep capture tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RhiConformanceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RhiConformanceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RhiConformanceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.15 and shared tests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.15 RHI conformance and backend parity

Architecture: capture tests

This page defines the architecture for RHI conformance and backend parity, with particular attention to capture tests. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Capture tests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with capability matrix is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for capture tests; distinguish required behavior from illustrative implementation detail.
- Keep capability matrix behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Rhi", "RhiConformanceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.15 and capture tests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.15 RHI conformance and backend parity

Data model: capability matrix

This page defines the data model for RHI conformance and backend parity, with particular attention to capability matrix. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Capability matrix is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with performance baseline is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for capability matrix; distinguish required behavior from illustrative implementation detail.
- Keep performance baseline behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RhiConformanceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RhiConformanceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RhiConformanceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.15 and capability matrix.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.15 RHI conformance and backend parity

Public API: performance baseline

This page defines the public api for RHI conformance and backend parity, with particular attention to performance baseline. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Performance baseline is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with shared tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for performance baseline; distinguish required behavior from illustrative implementation detail.
- Keep shared tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Rhi", "RhiConformanceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.15 and performance baseline.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.15 RHI conformance and backend parity

Ownership and lifetime: shared tests

This page defines the ownership and lifetime for RHI conformance and backend parity, with particular attention to shared tests. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Shared tests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with capture tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for shared tests; distinguish required behavior from illustrative implementation detail.
- Keep capture tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RhiConformanceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RhiConformanceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RhiConformanceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.15 and shared tests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.15 RHI conformance and backend parity

Threading model: capture tests

This page defines the threading model for RHI conformance and backend parity, with particular attention to capture tests. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Capture tests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with capability matrix is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for capture tests; distinguish required behavior from illustrative implementation detail.
- Keep capability matrix behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Rhi", "RhiConformanceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.15 and capture tests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.15 RHI conformance and backend parity

Memory strategy: capability matrix

This page defines the memory strategy for RHI conformance and backend parity, with particular attention to capability matrix. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Capability matrix is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with performance baseline is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for capability matrix; distinguish required behavior from illustrative implementation detail.
- Keep performance baseline behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RhiConformanceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RhiConformanceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RhiConformanceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.15 and capability matrix.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.15 RHI conformance and backend parity

Failure handling: performance baseline

This page defines the failure handling for RHI conformance and backend parity, with particular attention to performance baseline. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Performance baseline is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with shared tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for performance baseline; distinguish required behavior from illustrative implementation detail.
- Keep shared tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Rhi", "RhiConformanceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.15 and performance baseline.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.15 RHI conformance and backend parity

Diagnostics: shared tests

This page defines the diagnostics for RHI conformance and backend parity, with particular attention to shared tests. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Shared tests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with capture tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for shared tests; distinguish required behavior from illustrative implementation detail.
- Keep capture tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RhiConformanceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RhiConformanceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RhiConformanceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.15 and shared tests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.15 RHI conformance and backend parity

Implementation sequence: capture tests

This page defines the implementation sequence for RHI conformance and backend parity, with particular attention to capture tests. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Capture tests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with capability matrix is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for capture tests; distinguish required behavior from illustrative implementation detail.
- Keep capability matrix behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Rhi", "RhiConformanceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.15 and capture tests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.15 RHI conformance and backend parity

Verification: capability matrix

This page defines the verification for RHI conformance and backend parity, with particular attention to capability matrix. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Capability matrix is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with performance baseline is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for capability matrix; distinguish required behavior from illustrative implementation detail.
- Keep performance baseline behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RhiConformanceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RhiConformanceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RhiConformanceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.15 and capability matrix.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.15 RHI conformance and backend parity

Completion gate: performance baseline

This page defines the completion gate for RHI conformance and backend parity, with particular attention to performance baseline. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Performance baseline is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with shared tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for performance baseline; distinguish required behavior from illustrative implementation detail.
- Keep shared tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Rhi", "RhiConformanceAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 04.15 and performance baseline.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

04.15 RHI conformance and backend parity

Purpose and boundary: shared tests

This page defines the purpose and boundary for RHI conformance and backend parity, with particular attention to shared tests. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Shared tests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with capture tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for shared tests; distinguish required behavior from illustrative implementation detail.
- Keep capture tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RhiConformanceAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RhiConformanceAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RhiConformanceAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 04.15 and shared tests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.